

Shipping on the Air with Patterns

Assignment #02 - Software Architecture and Platforms

A.Y. 2025/2026

Fabio Vincenzi

7 agosto 2026

Indice

1	Introduzione	2
2	Quality Attribute Scenarios	2
3	Deployment	3
4	Patterns	3
4.1	API Gateway	3
4.2	Observability Patterns	4
4.2.1	Health Check API	4
4.2.2	Application Metrics	4
4.3	Event Sourcing	4
4.4	CQRS	6
4.5	Service Discovery	6
5	Testing	7
5.1	Unit	7
5.2	Integration	7
5.3	Component	8
5.4	End-to-end	8
5.5	Fitness function	8
6	Observability al servizio dei Quality Attribute Scenarios	9
6.1	QAS-A: Availability	9
6.2	QAS-B: Saturazione	9
7	Limiti noti	10
8	Conclusioni	10

1 Introduzione

Questo documento descrive, a livello architetturale, come il sistema *Shipping on the Air* progettato nell'Assignment 1 sia stato raffinato applicando un sottoinsieme dei microservices patterns discussi nel corso.

Il dominio e le funzionalità non cambiano: il sistema resta composto da **order-service**, **delivery-service** e **drone-service**, ai quali si aggiunge l'**api-gateway**. Ciò che cambia è il modo in cui i servizi comunicano con l'esterno, vengono osservati a runtime, persistono il proprio stato e vengono verificati.

Ogni pattern è presentato indicando il problema che risolve nel contesto specifico del progetto, la soluzione adottata e le conseguenze, incluse quelle negative. Dove possibile le affermazioni sono accompagnate da misure raccolte sul sistema in esecuzione.

Pattern applicati. Oltre ai tre richiesti dalla consegna (API Gateway, Observability Patterns e Event Sourcing applicato a un servizio) sono stati scelti **CQRS** e **Service Discovery**. La scelta non è arbitraria: come argomentato nella sezione 4.4, CQRS non è un'aggiunta facoltativa ma una conseguenza tecnica diretta dell'Event Sourcing.

2 Quality Attribute Scenarios

Gli attributi di qualità individuati nell'Assignment 1 sono stati riformulati in due scenari concreti, secondo la struttura in sei parti vista a lezione. Entrambi riguardano la stessa proprietà architetturale, la *degradazione controllata*, ma sotto due stimoli diversi: il guasto di un servizio e l'esaurimento di una risorsa.

Listing 1: QAS-A, Availability: guasto di un servizio a valle

Source	un guasto infrastrutturale
Stimulus	il drone-service diventa irraggiungibile
Environment	operazioni normali, sistema deployato in container
Artifact	delivery-service e piattaforma di deployment
Response	il guasto viene rilevato e il sistema continua ad accettare ordini e a programmare consegne, senza assegnare droni
Response Measure	rilevamento entro 15 s; nessun fallimento propagato al cliente

Listing 2: QAS-B, Saturazione: flotta di droni esaurita

Source	domanda superiore alla capacità del sistema
Stimulus	arriva un ordine quando tutti i droni sono occupati
Environment	operazioni normali, flotta satura
Artifact	delivery-service e drone-service
Response	l'ordine viene accettato e la consegna programmata, in attesa che un drone si liberi

Response Measure	<code>nessun ordine rifiutato; saturazione osservabile</code>
	<code>come drones_available = 0</code>

La verifica di questi scenari è discussa nella sezione 6, dopo aver introdotto gli observability pattern che li rendono misurabili.

3 Deployment

Il sistema adotta il pattern **Service per Container**: ogni microservizio viene impacchettato in un'immagine Docker separata e il sistema completo è descritto da un file Docker Compose.

Ogni servizio ha un **Dockerfile multi-stage**: un primo stage basato su JDK e Gradle esegue `installDist`, un secondo stage contiene solo il JRE e la distribuzione prodotta. L'immagine finale non contiene quindi né il JDK né il build tool. Il contesto di build è la radice del monorepo, perché tutti i servizi dipendono dal modulo `common`.

Il file `docker-compose.yml` definisce la rete logica `shipping_network`, alla quale tutti i servizi sono collegati, e imposta per ciascuno la policy `restart: unless-stopped` e un `healthcheck` (sezione 4.2.1). La configurazione dipendente dall'ambiente è iniettata tramite variabili d'ambiente, secondo il pattern **Externalized Configuration**: i launcher leggono host e porta dei servizi con cui devono comunicare, usando `localhost` come valore di fallback. Lo stesso artefatto funziona quindi sia in esecuzione locale sia in container, senza modifiche al codice.

4 Patterns

4.1 API Gateway

Problema. Senza un punto di ingresso unico, un client dovrebbe conoscere indirizzo e numero dei servizi, effettuare più chiamate per popolare una singola schermata e ogni servizio dovrebbe reimplementare le funzionalità trasversali.

Soluzione. È stato aggiunto il modulo `api-gateway` (porta 8080, prefisso `/api/v1`), unico componente esposto all'esterno. Il `GatewayController` instrada le richieste verso il servizio competente attraverso tre proxy (`OrderServiceProxy`, `DeliveryServiceProxy`, `DroneServiceProxy`), definiti come porte in uscita nell'application layer e implementati nell'infrastructure layer.

API Composition. L'endpoint di tracking `GET /api/v1/tracking/:orderId` non si limita a instradare: interroga l'`order-service` e il `delivery-service` e **unisce le due risposte** in un unico oggetto `TrackingInfo`. È un'applicazione del pattern *API Composition*, necessario perché con un database per servizio non è possibile eseguire join tra dati di servizi diversi. Il client effettua una chiamata sola invece di due.

Poiché i proxy sono sincroni e bloccanti, le chiamate a valle sono eseguite tramite `executeBlocking` con `ordered = false`, in modo da non bloccare l'event loop di Vert.x.

Conseguenze. Il gateway semplifica i client e disaccoppia il loro codice dalla struttura interna del sistema, ma introduce un componente che deve essere reso altamente disponibile e che va aggiornato ogni volta che cambia un'API a valle.

4.2 Observability Patterns

4.2.1 Health Check API

Tutti e quattro i componenti espongono un endpoint `/health` che restituisce lo stato del servizio nel formato MicroProfile Health (`status` e lista di `checks`).

L'endpoint non serve solo alla diagnosi manuale: è consumato dall'attributo `healthcheck` del Docker Compose file, che lo interroga periodicamente e marca il container come `unhealthy` in caso di fallimento. Combinato con la policy di restart, questo permette alla piattaforma di reagire autonomamente a un servizio bloccato.

4.2.2 Application Metrics

Ogni componente espone metriche in formato Prometheus su una **porta dedicata**, distinta da quella di servizio, tramite l'`HTTPServer` della libreria `io.prometheus`. Un container `prometheus` raccoglie i quattro target con `scrape_interval: 10s`.

Componente	Metrica	Tipo e significato
order-service	<code>orders_placed_total</code>	counter: ordini creati
delivery-service	<code>deliveries_in_progress</code>	gauge: consegne in corso
drone-service	<code>drones_available</code>	gauge: droni liberi
api-gateway	<code>gateway_requests_total</code>	counter: richieste ricevute

La scelta del tipo di metrica riflette la natura del fenomeno osservato: un *counter* per eventi cumulativi che possono solo crescere, un *gauge* per quantità che salgono e scendono. Sui counter va sempre usato `rate()`, poiché il valore assoluto si azzerà al riavvio del processo.

Come sono raccolte. Per i tre servizi di dominio la strumentazione non è inserita nei controller ma realizzata tramite **observer**: l'application layer definisce una porta in uscita (ad esempio `DeliveryServiceObserver`) e l'infrastructure layer ne fornisce un adapter (`PrometheusDeliveryServiceObserver`). Il dominio non conosce quindi Prometheus e resta osservabile senza esserne dipendente. Il gateway costituisce un'eccezione consapevole: non avendo un core di dominio, la sua metrica è puramente infrastrutturale e viene incrementata da un handler di routing.

4.3 Event Sourcing

Scelta del servizio. L'Event Sourcing è stato applicato al `delivery-service`. È un pattern invasivo, che richiede di ristrutturare la business logic, quindi va applicato dove produce valore: la consegna ha un ciclo di vita a stati articolato (`SCHEDULED` → `DRONE_ASSIGNED` → `IN_TRANSIT` → `DELIVERED/FAILED`) e una storia che ha valore di per sé: dove si trovava il drone in un dato istante, quando è stato assegnato. Applicarlo anche agli altri servizi avrebbe aggiunto complessità senza beneficio.

Ristrutturazione dell'aggregate. Ogni metodo di comando è stato separato in due:

- i metodi `processXxx()` validano la richiesta, **non modificano lo stato** e restituiscono l'evento da applicare; sono gli unici che possono fallire;
- i metodi `apply()` aggiornano lo stato a partire da un evento e non possono fallire, perché un evento descrive un fatto già accaduto.

Tutte e sei le transizioni emettono ora un evento (`DeliveryScheduled`, `DroneAssigned`, `TransitStarted`, `DronePositionUpdated`, `DeliveryCompleted`, `DeliveryFailed`); in precedenza solo due lo facevano.

Una conseguenza non ovvia riguarda il **determinismo**: poiché lo stato viene ricostruito riapplicando gli eventi, i metodi `apply()` non possono contenere sorgenti di non-determinismo. L'istante di creazione, prima calcolato con `Instant.now()` nel costruttore, è stato spostato **dentro** l'evento `DeliveryScheduled`, in modo che ogni replay riproduca esattamente lo stesso stato.

Event store. La porta `DeliveryEventStore` espone `append`, `load` e `loadFrom`: non esiste alcuna operazione di modifica o cancellazione, quindi l'append-only è garantito dalla forma dell'interfaccia e non dalla disciplina di chi la usa.

Il metodo `append` riceve una `expectedVersion`, cioè la versione a cui il chiamante ha letto l'aggregate, e rifiuta la scrittura se lo stream è nel frattempo avanzato. È un controllo di **concorrenza ottimistica** che impedisce a due comandi concorrenti di crederci entrambi l'ultimo, e costituisce la differenza sostanziale tra un event store e una semplice coda di messaggi.

Snapshot. Poiché il `delivery-service` riceve aggiornamenti di posizione ad alta frequenza, lo stream di una consegna cresce indefinitamente e il costo di ogni lettura crescerebbe con esso. Viene quindi persistito uno snapshot dello stato ogni 50 eventi: il caricamento parte dallo snapshot più recente e riapplica soltanto gli eventi successivi.

Misurato in locale su uno stream di 1003 eventi, con media su 2000 caricamenti dopo warm-up:

	Tempo per <code>findById</code>
senza snapshot (replay di 1003 eventi)	0,013 ms
con snapshot (replay dei soli eventi successivi)	0,004 ms

Il guadagno misurato è di circa $3,6\times$. Il rapporto è contenuto perché event store e snapshot sono in memoria e gli eventi hanno un payload minimo, quindi si sta misurando il solo costo del *fold*; con una persistenza su database il divario crescerebbe, dovendo leggere 1003 righe invece di uno snapshot e pochi eventi. Il punto architetturale, comunque, non è il fattore ma il fatto che il costo passi da $O(\text{numero di eventi})$ a $O(\text{eventi dopo lo snapshot})$, cioè da crescente a costante.

Lo snapshot è **stato derivato**: eliminarlo lascia il sistema corretto, solo più lento. La proprietà è verificata dal test `droppingEverySnapshotKeepsTheStateCorrect`, che carica la stessa consegna con e senza snapshot e confronta i due stati risultanti.

4.4 CQRS

Perché è necessario. Un event store risponde soltanto per chiave primaria, cioè per identificatore di consegna. Il `DeliveryRepository`, però, espone anche `findByOrderId` e `findByDroneId`: due interrogazioni su campi che chiave non sono. Dopo l'introduzione dell'Event Sourcing queste operazioni non sarebbero più implementabili se non ricostruendo l'intero insieme delle consegne a ogni richiesta.

CQRS non è quindi un pattern aggiunto per scelta estetica, ma la conseguenza diretta della decisione presa nella sezione 4.3: **è l'Event Sourcing a imporlo.**

Soluzione. I due lati sono separati anche strutturalmente:

- il **write model** è costituito dall'aggregate e dall'event store, append-only e fonte di verità;
- il **read model** è la porta `DeliveryLookupView` con il relativo adapter, che mantiene due indici `orderId → deliveryId` e `droneId → deliveryId`;
- il `DeliveryProjector` traduce gli eventi in aggiornamenti della vista ed è **l'unico componente autorizzato a scriverci.**

Il repository scrive attraverso il projector e legge attraverso la vista: i comandi non toccano mai il read model e le interrogazioni non attraversano mai il modello di dominio.

La vista contiene deliberatamente i soli identificatori e non lo stato della consegna. In questo modo l'unica informazione duplicata è una corrispondenza che non cambia mai nel tempo: nel peggiore dei casi una consegna non viene trovata, mai vengono restituiti dati errati.

Ricostruibilità. Il metodo `rebuildLookupView()` svuota la vista e la ricostruisce riapplicando tutti gli eventi memorizzati, usando la stessa funzione di proiezione impiegata a regime. Anche il read model, come lo snapshot, è dato derivato: il test `theViewCanBeDroppedAndRebuilt` lo dimostra.

Consistenza. Nell'implementazione realizzata la proiezione è invocata in modo sincrono subito dopo la scrittura degli eventi, quindi la vista è sempre allineata e non si manifesta il problema del *read-your-own-writes*. In cambio non si ottiene il disaccoppiamento completo: il passaggio a una proiezione asincrona richiederebbe di spostare la chiamata al projector dietro una sottoscrizione all'event store, **senza modificare né la vista né il repository**, proprio perché la vista è una porta separata.

4.5 Service Discovery

Problema. In un deployment a container gli indirizzi dei servizi sono dinamici, quindi non possono essere scritti nel codice né in un file di configurazione statico.

Soluzione. Il pattern è realizzato **a livello di deployment**: Docker fornisce nativamente un service registry e un DNS resolver interno, assegnando a ogni servizio un nome sulla rete `shipping_network`. Si tratta del modello *server-side discovery*: il chiamante usa sempre lo stesso nome logico e l'infrastruttura risolve verso un'istanza attiva.

Un dettaglio determinante è che le variabili d'ambiente iniettate dal Compose file puntano ai **nomi di servizio** (`DELIVERY_HOST=delivery-service`) e non ai `container_name` (`delivery-01`). È la differenza tra discovery effettiva e indirizzo fisso: il nome di servizio viene risolto dinamicamente e, in presenza di più repliche, distribuito tra tutte; il nome del container punterebbe sempre alla stessa istanza.

Il pattern si integra con l'Externalized Configuration: lo stesso artefatto usa `localhost` in esecuzione locale e il nome logico in container. I limiti attuali di questa configurazione sono discussi nella sezione 7.

5 Testing

La strategia di testing segue la test pyramid, con almeno un esempio per ciascun livello. I livelli sono distinti nella struttura dei package, in modo che la piramide sia leggibile dall'organizzazione del codice e non solo da questa descrizione.

5.1 Unit

Gli unit test sono organizzati per strato dell'architettura esagonale, seguendo la distinzione tra test *solitary* (dipendenze sostituite da mock) e *sociable* (dipendenze reali).

- **application:** `DeliveryServiceTest` verifica che il completamento di una consegna notifichi l'order-service e rilasci il drone; le porte verso gli altri servizi sono sostituite da stub. È un test *solitary*.
- **infrastructure:** `DeliveryEventStoreTest`, `DeliverySnapshotTest` e `DeliveryProjectionTest` verificano gli adapter di persistenza (append-only, ordinamento, rifiuto delle scritture concorrenti, replay, snapshot e ricostruzione della vista) usando gli oggetti di dominio reali. Sono test *sociable*.
- **controller:** `DeliveryControllerTest` verifica rotte, parsing del payload, codici di stato e formato JSON, con un `DeliveryServiceMock` al posto del servizio. È un test *solitary*: il soggetto è una sola classe, e la porta HTTP è soltanto il modo in cui un adapter di questo tipo può essere invocato.

Un test merita una nota particolare: `DeliveryEventStoreTest` verifica che l'istante di creazione sopravviva identico a un replay, proteggendo così la proprietà di determinismo descritta nella sezione 4.3.

5.2 Integration

`DroneServiceProxyTest` verifica il contratto tra due servizi: il `DroneController` reale del drone-service viene avviato su una porta di test con un servizio applicativo reale su repository in memoria, e vi si punta contro il `DroneServiceProxy` reale del delivery-service. Ciò che viene verificato non è la logica di business, ma che URL, query parameter, formato JSON e codici di stato dei due lati coincidano.

Essendo il progetto un monorepo Gradle, il test dichiara semplicemente `testImplementation project(':drone-service')`: non è stato necessario duplicare classi tra i moduli, e la dipendenza esiste soltanto in ambito di test.

Un caso di questo gruppo verifica una proprietà rilevante per la sezione 6: con il servizio a valle irraggiungibile, il proxy restituisce un risultato vuoto anziché propagare l'eccezione.

5.3 Component

Gli acceptance test del singolo servizio sono scritti in Gherkin ed eseguiti con Cucumber. Il delivery-service viene avviato **per intero** (controller, application layer e persistenza a eventi) su una porta di test, e gli step lo interrogano **attraverso la sua API HTTP**, come farebbe un client. Solo le porte verso gli altri servizi sono sostituite da stub.

Il confine del test è quindi l'API pubblica del servizio, non le sue classi: gli step manipolano oggetti JSON e non hanno accesso al modello di dominio. Ne consegue che una modifica interna non rompe questi test, mentre una modifica al contratto pubblico sì, che è esattamente il comportamento desiderato per un acceptance test.

Si tratta di component test *in-process*: la rete e il protocollo HTTP sono reali, mentre la verifica del packaging in container è demandata al livello end-to-end.

5.4 End-to-end

Il modulo separato **system-end-to-end** contiene uno user journey che attraversa l'intero sistema passando esclusivamente dall'API gateway: registrazione di un drone e di un utente, creazione di un ordine, conferma, e verifica che il tracking riporti una consegna con un drone assegnato. Il percorso esercita tutti e quattro i componenti e l'API Composition del gateway.

Poiché richiede il sistema in esecuzione, questo modulo è escluso dalla build ordinaria ed è attivato da una proprietà esplicita:

Listing 3: Esecuzione dei test end-to-end

```
docker compose up -d
./gradlew :system-end-to-end:test -Pe2e
```

5.5 Fitness function

Oltre ai quattro livelli, i tre servizi di dominio (**order-service**, **delivery-service** e **drone-service**) includono un **ArchitectureTest** basato su ArchUnit, che verifica che il dominio non dipenda da application e infrastructure, che le porte risiedano nell'application layer e che gli adapter risiedano nell'infrastructure layer. Non appartiene alla test pyramid: è una *fitness function*, cioè un controllo automatico di una caratteristica architetturale, e come tale uno strumento di governance dell'architettura.

L'**api-gateway** ne è privo, per una ragione che vale la pena esplicitare: non avendo un core di dominio da proteggere (è un componente di sola infrastruttura, come discusso nella sezione 4.2.2 a proposito delle metriche), la regola di dipendenza che le fitness function verificano negli altri servizi non ha qui un equivalente.

6 Observability al servizio dei Quality Attribute Scenarios

Un Quality Attribute Scenario è verificabile solo se la sua *Response Measure* è osservabile. Senza strumentazione uno scenario resta un'intenzione: è questo il ruolo degli observability pattern rispetto ai requisiti di qualità. La relazione è analoga a quella delle fitness function, con la differenza che queste verificano caratteristiche architetturali a tempo di build, mentre l'observability le verifica a runtime.

Entrambi gli scenari della sezione 2 sono stati tradotti in test eseguibili (`QualityAttributeScenario`) che provocano la condizione avversa e leggono le metriche.

6.1 QAS-A: Availability

Gli observability pattern coinvolti sono due. L'**Health Check API** rende interrogabile lo stato di ciascun servizio; le **Application Metrics** lo trasformano in una serie temporale, perché Prometheus genera automaticamente per ogni target la metrica `up`, pari a 1 finché lo scrape riesce e 0 appena fallisce. È il punto in cui i due pattern si saldano.

Il test arresta il container del drone-service, attende che la metrica cambi e verifica il comportamento del sistema in quello stato:

Listing 4: Esito osservato

```
up{job="monitoring-drone-service"} = 0      (gli altri tre restano
a 1)
guasto rilevato in 8,157 s

ordine creato -> PENDING
confirm       -> HTTP 200
tracking      -> hasDelivery: true, deliveryStatus: "SCHEDULED",
                droneId: null
```

La consegna viene programmata comunque, semplicemente priva di drone: il guasto non si propaga al cliente. Il tempo di rilevamento misurato è coerente con lo `scrape_interval` di 10 s: la latenza di rilevamento è determinata dalla frequenza di campionamento, non dal servizio. La *Response Measure* dichiarata (entro 15 s) è quindi soddisfatta, ma è importante osservare che essa dipende da una scelta di configurazione dell'infrastruttura di monitoraggio.

È utile notare come questa proprietà sia coperta due volte, in due modi complementari: il test di integrazione `anUnreachableServiceYieldsNoDrone` la **garantisce** prima del deploy, verificando che il proxy restituisca un risultato vuoto; la metrica la **osserva** in esercizio.

6.2 QAS-B: Saturazione

Qui l'observability pattern rilevante è la sola Application Metrics, attraverso il gauge `drones_available`. La scelta del tipo di metrica è essa stessa parte della soluzione: un gauge può salire e scendere ed è quindi adatto a rappresentare una risorsa esauribile, mentre un counter potrebbe solo indicare quanti droni siano stati registrati complessivamente.

Il test crea e conferma ordini finché il gauge non raggiunge lo zero, quindi effettua un ulteriore ordine e verifica che venga comunque accettato e programmato, in attesa di un drone. La forma della risposta è la stessa del QAS-A, cioè `SCHEDULED` con `droneId` nullo, ma la causa è l'esaurimento di una risorsa anziché un guasto.

7 Limiti noti

I servizi non sono replicabili orizzontalmente. La configurazione attuale prevede un'istanza per servizio. Tentando di avviare più repliche emergono tre ostacoli, di gravità crescente: i `container_name` espliciti impediscono lo scaling, perché i nomi devono essere univoci; la pubblicazione di porte fisse sull'host causa un conflitto di binding; e soprattutto i servizi mantengono lo stato in repository in memoria, per cui repliche diverse avrebbero stati divergenti. I primi due ostacoli sono scelte di leggibilità in fase di sviluppo, facilmente reversibili; il terzo è strutturale. La strada per lo scaling passa dall'estrarre lo stato in uno store condiviso, operazione che nel `delivery-service` richiederebbe di sostituire il solo adapter, dato che l'event store è già definito dietro una porta.

La persistenza è in memoria. L'event store, gli snapshot e la vista di lettura sono implementati con strutture dati in memoria, coerentemente con gli esempi visti a lezione. Ne consegue che la storia degli eventi non sopravvive al riavvio di un container, il che è in tensione con uno dei benefici dell'Event Sourcing, l'audit trail permanente. Anche in questo caso il costo del cambiamento è limitato alla sostituzione dell'adapter.

Non tutti i QAS sono misurabili. Lo scenario di performance formulato nell'Assignment 1 (tracking con latenza inferiore al secondo) non è verificabile con la strumentazione attuale, composta da counter e gauge, che indicano quante richieste siano state servite ma non quanto tempo abbiano impiegato. Servirebbe un istogramma delle durate sul gateway, dal quale ricavare i percentili. La conseguenza generale merita di essere esplicitata: **lo strumento di misura determina quali attributi di qualità è possibile promettere.**

Event Sourcing parziale. L'event store realizzato copre il lato database della definizione, ossia l'inserimento e il recupero degli eventi per chiave, ma non espone un meccanismo di sottoscrizione: le notifiche verso gli altri servizi restano effettuate esplicitamente dall'application layer. Non sono inoltre affrontate l'evoluzione dello schema degli eventi né la cancellazione dei dati personali, entrambe rilevanti in un sistema destinato alla produzione.

8 Conclusioni

L'applicazione dei pattern ha modificato il sistema lungo tre direzioni distinte.

Verso l'esterno, l'API Gateway ha reso il sistema utilizzabile attraverso un unico punto di ingresso, nascondendo ai client la decomposizione interna; l'API Composition ha reso possibile una query che attraversa due servizi, altrimenti impraticabile con un database per servizio.

A runtime, gli observability pattern hanno trasformato requisiti di qualità in proprietà misurabili. È l'aspetto più significativo del lavoro: senza strumentazione, uno scenario

come “il guasto di un servizio non deve propagarsi” non sarebbe stato né verificabile né dimostrabile, mentre è stato possibile misurarne il rilevamento in 8,157 s e osservare il sistema continuare a operare in modo degradato.

Sulla persistenza, l'Event Sourcing ha sostituito la memorizzazione dello stato con quella della sua storia, ottenendo un registro immutabile e la possibilità di ricostruire lo stato a qualunque istante, al prezzo di una ristrutturazione della business logic e della necessità di introdurre CQRS per le interrogazioni non basate su chiave. Quest'ultimo punto è forse l'insegnamento più utile dell'intero assignment: i pattern non sono elementi indipendenti da collezionare, ma soluzioni che si richiamano l'una con l'altra, e adottarne uno significa spesso accettarne un secondo.

Infine, la test pyramid ha permesso di distribuire la verifica su livelli con obiettivi diversi, dalla singola classe fino allo user journey attraverso il sistema completo, e di collegare i test alle metriche: le stesse proprietà garantite dai test prima del deploy sono osservate dalle metriche durante l'esercizio.